

Program 7.2: Object Eligibility for Garbage Collection

```
class Demo {  
    int id;  
    Demo(int id) {  
        this.id = id;  
    }  
}  
  
public class ObjectEligibility {  
    public static void main(String[] args) {  
        Demo d1 = new Demo(1);  
        d1 = null;  
        Demo d2 = new Demo(2);  
        d2 = new Demo(3);  
        new Demo(4);  
        System.gc();  
        System.out.println("Objects are eligible for garbage collection.");  
    }  
}
```

Output:

Objects are eligible for garbage collection.

Experiment 8: Method Overloading

Program 8.1: Overloading Arithmetic Methods

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

```

    double add(double a, double b) {
        return a + b;
    }
}

public class ArithmeticOverloading {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println("Two integers: " + c.add(10, 20));
        System.out.println("Three integers: " + c.add(10, 20, 30));
        System.out.println("Two doubles: " + c.add(10.5, 20.5));
    }
}

```

Output:

Two integers: 30
 Three integers: 60
 Two doubles: 31.0

Program 8.2: Overloading Area Methods

```

class Area {
    double area(double radius) {
        return Math.PI * radius * radius;
    }
    int area(int length, int width) {
        return length * width;
    }
    int area(int side) {
        return side * side;
    }
}

public class AreaOverloading {

```

```

public static void main(String[] args) {
    Area a = new Area();
    System.out.println("Circle Area: " + a.area(5.0));
    System.out.println("Rectangle Area: " + a.area(10, 5));
    System.out.println("Square Area: " + a.area(4));
}
}

```

Output:

Circle Area: 78.53981633974483

Rectangle Area: 50

Square Area: 16

Experiment 9: Using Objects as Parameters

Program 9.1: Passing Student Object

```

class Student {
    String name;
    int rollNo;
    Student(String name, int rollNo) {
        this.name = name;
        this.rollNo = rollNo;
    }
}

public class StudentParameter {
    static void display(Student s) {
        System.out.println("Name: " + s.name);
        System.out.println("Roll Number: " + s.rollNo);
    }

    public static void main(String[] args) {
        Student s = new Student("Raina", 101);
        display(s);
    }
}

```

```
}  
}
```

Output:

Name: Raina

Roll Number: 101

Program 9.2: Comparing Employee Salaries

```
class Employee {  
    String name;  
    double salary;  
    Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
}  
  
public class EmployeeSalary {  
    static void compare(Employee e1, Employee e2) {  
        if (e1.salary > e2.salary) {  
            System.out.println(e1.name + " has the higher salary.");  
        } else if (e2.salary > e1.salary) {  
            System.out.println(e2.name + " has the higher salary.");  
        } else {  
            System.out.println("Both employees have the same salary.");  
        }  
    }  
}  
  
public static void main(String[] args) {  
    Employee e1 = new Employee("Rahul", 50000);  
    Employee e2 = new Employee("Ananya", 60000);  
    compare(e1, e2);  
}
```

```
}
```

Output:

Ananya has the higher salary.

Experiment 10: Returning Objects

Program 10.1: Returning a Student Object

```
class Student {  
    String name;  
    int rollNo;  
    Student(String name, int rollNo) {  
        this.name = name;  
        this.rollNo = rollNo;  
    }  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Roll Number: " + rollNo);  
    }  
}  
  
public class ReturnStudent {  
    static Student createStudent() {  
        Student s = new Student("Raina", 101);  
        return s;  
    }  
    public static void main(String[] args) {  
        Student s = createStudent();  
        s.display();  
    }  
}
```

Output:

Name: Raina

Roll Number: 101

Program 10.2: Returning a Bank Account Object

```
class BankAccount {  
    String name;  
    double balance;  
    BankAccount(String name, double balance) {  
        this.name = name;  
        this.balance = balance;  
    }  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Balance: " + balance);  
    }  
}  
  
public class ReturnBankAccount {  
    static BankAccount updateBalance(BankAccount account) {  
        account.balance += 5000;  
        return account;  
    }  
    public static void main(String[] args) {  
        BankAccount b = new BankAccount("Ravi", 20000);  
        b = updateBalance(b);  
        b.display();  
    }  
}
```

Output:

Name: Ravi

Balance: 25000.0

Experiment 11: Understanding static

Program 11.1: Static Variable

```
class Student {  
    String name;  
    static int count = 0;  
    Student(String name) {  
        this.name = name;  
        count++;  
    }  
    void display() {  
        System.out.println("Name: " + name);  
    }  
}  
  
public class StaticVariable {  
    public static void main(String[] args) {  
        Student s1 = new Student("Raina");  
        Student s2 = new Student("Rinku");  
        Student s3 = new Student("Amar");  
        s1.display();  
        s2.display();  
        s3.display();  
        System.out.println("Total Students: " + Student.count);  
    }  
}
```

Output:

Name: Raina